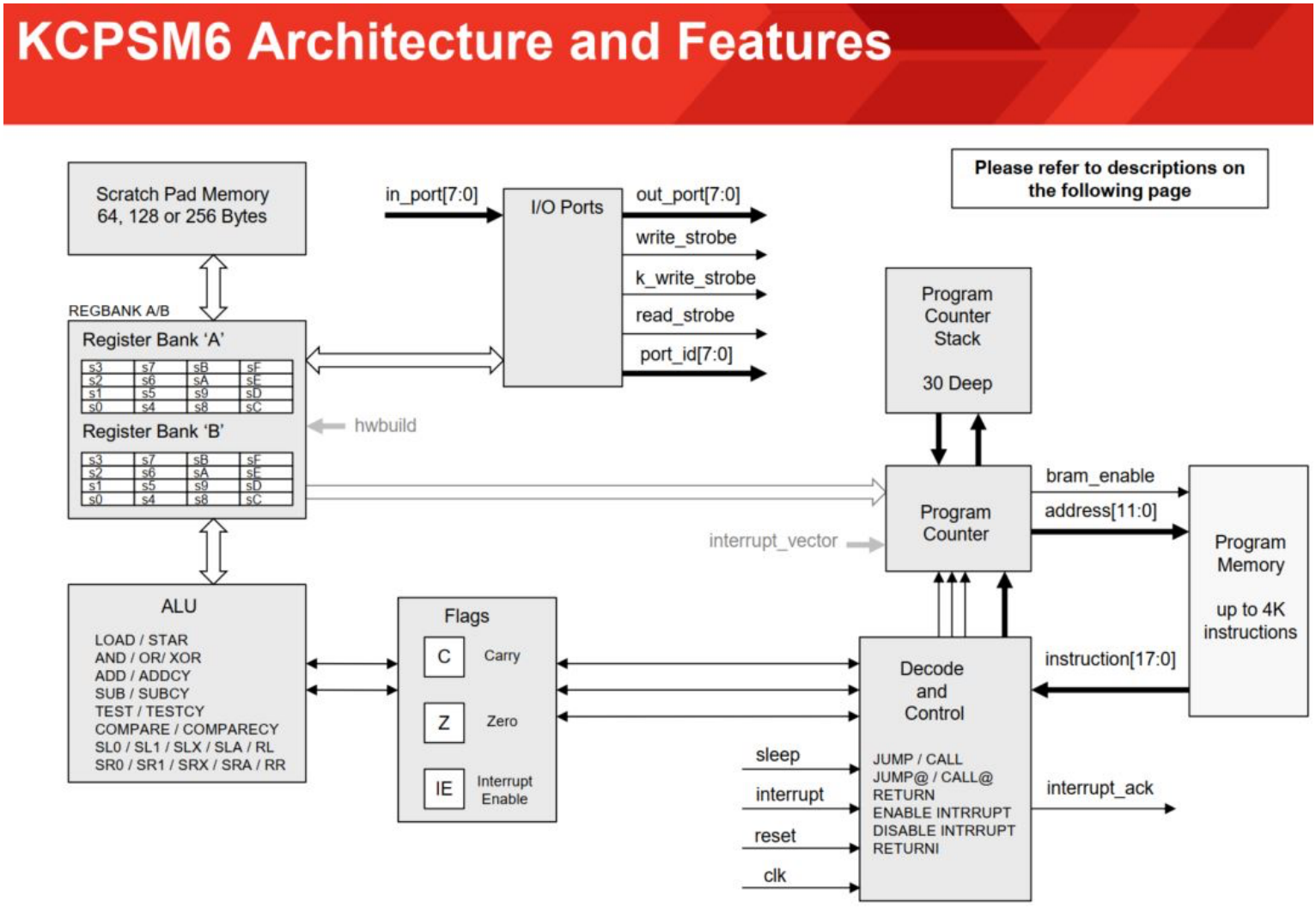
Embedded a Soft-Core

1. Pico Blaze Basics

- Logical instructions
- Arithmetic instructions
- Compare and test instructions
- Shift and rotate instructions
- Data movement instructions
- Program flow control instructions
- Interrupt related instructions

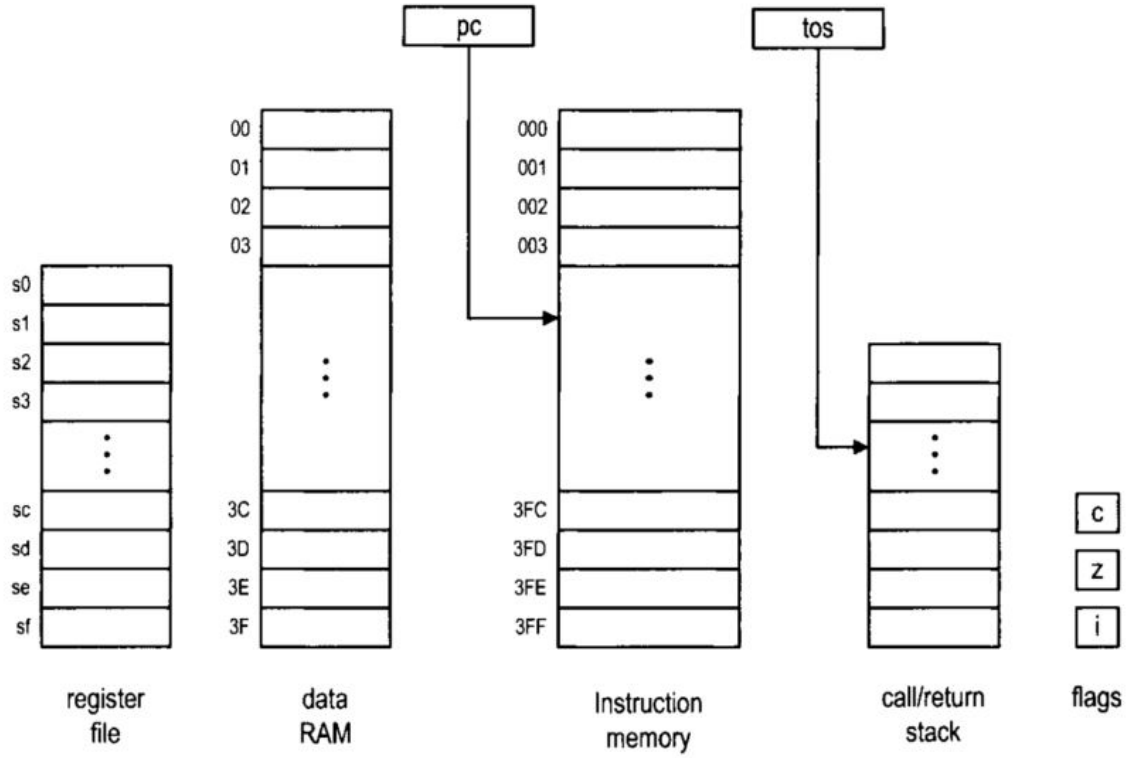
The architecture and the instruction set introduced below are for KCPSM3. For the difference between these two versions, you can refer to the official KCPSM6 User Guide.

The Xilinx PicoBlaze User Guide has more details about the circuit diagram



The following snapshots are from Pong Chu's book: FPGA Prototyping By Verilog Examples - Xilinx Spartan - 3 Version. The contents may be for the old version KCPSM3 but it is very similar to

- 8-bit data width
- 8-bit ALU with carry and zero flags
- 16 8-bit general-purpose registers
- 64-byte data memory
- 18-bit instruction width
- 10-bit instruction address, which supports a program up to 1024 instructions
- 31-word call/return stack
- 256 input ports and 256 output ports
- 2 clock cycles per instruction
- 5 clock cycles for interrupt handling



We use the following notations for these memory components and some constant definitions:

- `sX`, `sY`: each representing one of the 16 general-purpose registers, where `X` and `Y` take on hexadecimal values from `0` to `f`
- `pc`: program counter
- `tos`: top-of-stack pointer of the call/return stack
- `c`, `z`, `i`: carry, zero, and interrupt flags
- `KK`: 8-bit constant value or port id, which is usually expressed as two hexadecimal digits
- `SS`: 6-bit constant data memory address, which is usually expressed as two hexadecimal digits
- `AAA`: 10-bit constant instruction memory address, which is usually expressed as three hexadecimal digits

- **op** `sX`, `sY`: *register-register format*. The **op** term specifies the operation. The `sX` and `sY` terms are the two operands and `sX` also serves as the destination register. It performs the `sX` ← `sX` **op** `sY` operation.
- **op** `sX`, `KK`: *register-constant format*. This format is similar to the register-register format except that the second operand is replaced by an immediate constant. It performs the `sX` ← `sX` **op** `KK` operation.
- **op** `sX`: *single-register format*. This format is used in shift and rotate instructions, which involve only one operand. It performs the `sX` ← **op** `sX` operation.
- **op** `AAA`: *single-address format*. This format is used in jump and call instructions. The `AAA` term is an address of the instruction memory. If the specified condition is met, `AAA` is loaded into the program counter.
- **op**: *zero-operand format*. This format is used in some miscellaneous instructions that do not involve any operand.

There are two assembler programs for PicoBlaze: KCPSM3 (we will use the newer release - KCPSM6) from Xilinx and PBlazeIDE from Mediatronix.

Logic Instructions:

There are six logical instructions, which support the and, or, and xor operations. An instruction performs bitwise logical operation between two registers or between one register and a constant. **The carry flag, c, is always cleared.** The zero flag, z, reflects the result ofthe operation. The mnemonics, brief descriptions, and pseudo operations of these instructions are:

- **and** sX, sY
  - bitwise and operation
  - pseudo operation:

```
sX ← sX & sY;
c ← 0;
```

- **and** sX, KK
  - bitwise and operation
  - pseudo operation:

```
sX ← sX & KK;
c ← 0;
```

- **or** sX, sY
  - bitwise or operation
  - pseudo operation:

```
sX ← sX | sY;
c ← 0;
```

- **or** sX, KK
  - bitwise or operation
  - pseudo operation:

```
sX ← sX | KK;
c ← 0;
```

- **xor** sX, sY
  - bitwise xor operation
  - pseudo operation:

```
sX ← sX ^ sY;
c ← 0;
```

- **xor** sX, KK
  - bitwise xor operation
  - pseudo operation:

```
sX ← sX ^ KK;
c ← 0;
```

Arithmetic Instructions:

There are eight arithmetic instructions, which support addition and subtraction with or without the carry flag. The carry flag, c, and the zero flag, z, reflect the result of operation. The mnemonics, brief descriptions, and pseudo operations of these instructions are:

- **add** sX, sY
  - add without the carry flag
  - pseudo operation:

```
sX ← sX + sY;
```

- **add** sX, KK
  - add without the carry flag
  - pseudo operation:

```
sX ← sX + KK;
```

- **addcy** sX, sY (**addc** sX, sY)
  - add with the carry flag
  - pseudo operation:

```
sX ← sX + sY + c;
```

- **addcy** sX, KK (**addc** sX, KK)
  - add with the carry flag
  - pseudo operation:

```
sX ← sX + KK + c;
```

- **sub** sX, sY
  - subtract without the carry flag
  - pseudo operation:

```
sX ← sX - sY;
```

- **sub** sX, KK
  - subtract without the carry flag
  - pseudo operation:

```
sX ← sX - KK;
```

- **subcy** sX, sY (**subc** sX, sY)
  - subtract with the carry flag (flag functioning as a borrow bit)
  - pseudo operation:

```
sX ← sX - sY - c;
```

- **subcy** sX, KK (**subc** sX, KK)
  - subtract with the carry flag (flag functioning as a borrow bit)
  - pseudo operation:

```
sX ← sX - KK - c;
```

Compare and Test Instructions:

sX == sY, zc: 10
sX < sY, zc: 01

sX > sY, zc: 00

- **compare** sX, sY (**comp** sX, sY)
  - compare two registers and set the flags
  - pseudo operation:

```
if sX==sY then z ← 1 else z ← 0;
if sY>sX then c ← 1 else c ← 0;
```

- **compare** sX, KK (**comp** sX, KK)
  - compare a register and a constant and set the flags
  - pseudo operation:

```
if sX==KK then z ← 1 else z ← 0;
if KK>sX then c ← 1 else c ← 0;
```

A test instruction performs an and operation. The result is used to set the flags and is not stored in any register. If the result is 0, the zero flag is set to 1. The result is also fed to an eight-input xor circuit to obtain the odd parity. If there are an odd number of 1's in the result, the carry flag is set to 1. The mnemonics, brief descriptions, and pseudo operations of the two instructions are shown below. The t is the 8-bit temporary result and will be discarded.

- **test** sX, sY
  - test two registers and set the flags
  - pseudo operation:

```
t ← sX & sY;
if t==0 then z ← 1 else z ← 0;
c ← t[7] ^ t[6] ^ ··· ^ t[0];
```

- **test** sX, KK
  - test a register and a constant and set the flags
  - pseudo operation:

```
t ← sX & KK;
if t==0 then z ← 1 else z ← 0;
c ← t[7] ^ t[6] ^ ··· ^ t[0];
```

To understand the bottom level structure of the Test instruction, refer to Page 63 of the KCPSM6 manual:

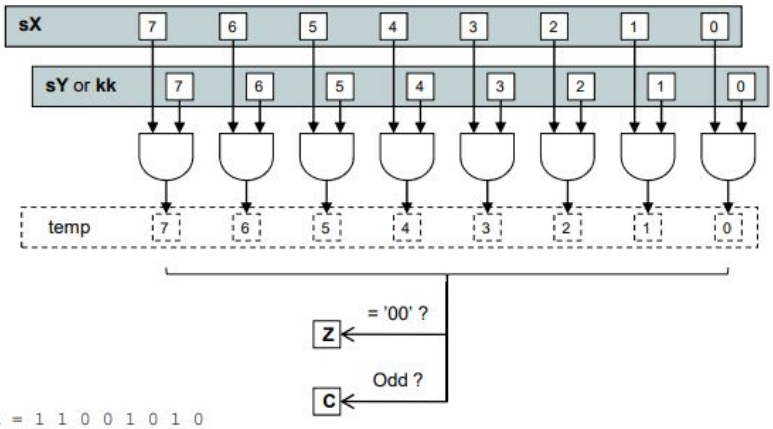
TEST sX, kk
TEST sX, sY

The 'TEST' instructions are similar to the 'AND' instructions in that a bit-wise logical AND operation is performed. However, the actual result is discarded and only the flags are updates to reflect the temporary 8-bit. The 'TEST' instruction also reports then exclusive-OR of the temporary result which can be used to compute the 'odd parity' of a value.

The first operand must specify a register 'sX' whose value provides one input to the AND operation (sX will not be effected by the operation).
The second operand defines the second input to the AND operation and can either be an 8-bit constant 'kk' or a register 'sY'.
The zero flag (Z) will be set if all 8-bits of the temporary result are zero.
The carry flag (C) will be set if the temporary result contains an odd number of bits set to '1' (the exclusive-OR of the 8-bit temporary result).

TEST sX, kk temp = sX AND kk

TEST sX, sY temp = sX AND sY



Hints

It is typical to think of 'sX' containing the information to be tested and for 'sY' or 'kk' to be acting as a bit-mask to select only those bits to be tested.

To test a single bit the value of 'kk' is best described using a binary format such as 00100000'b that will text bit 5 (equivalent to 20 hex). The 'C' flag will be set if the corresponding bit in 'sX' is '1' and the 'Z' flag will be set if the tested bit is '0'.

Use a mask value of kk = FF to compute the odd parity of the whole byte contained in 'sX'.

Examples

```
LOAD sA, CA
TEST sA, 01000000'b
```
Z=0, C=1 (odd).

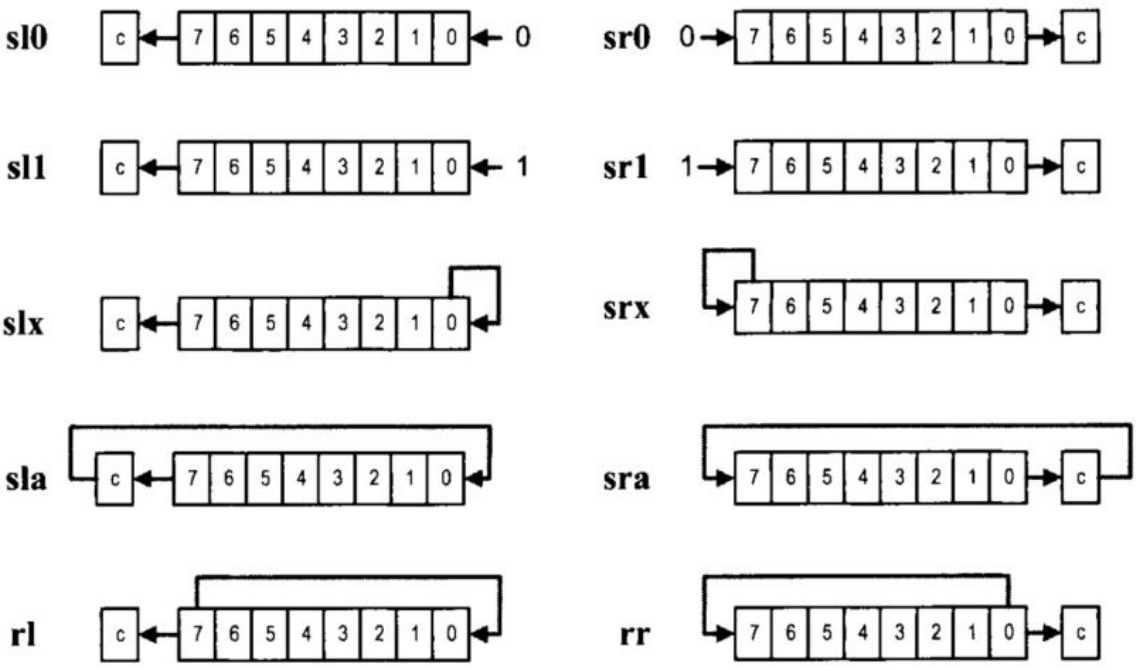
```
CA = 1 1 0 0 1 0 1 0
40 = 0 1 0 0 0 0 0 0
CA AND 40 = 0 1 0 0 0 0 0 0 = 40
```

```
LOAD sA, 51
TEST sA, FF
```
Z=0, C=1.
Parity is odd.

```
51 = 0 1 0 1 0 0 0 1
FF = 1 1 1 1 1 1 1 1
51 AND FF = 0 1 0 1 0 0 0 1 = 51
```

Hint – The 'SLA' and 'SRA' shift instructions and the ADDCY and SUBCY instructions can all be used to move the value of the carry flag into a register.

Shift and Rotate Instructions:



- **sl0** sX
  - shift a register left 1 bit and shift 0 into the LSB
  - pseudo operation:

```
sX ← {sX[6:0], 0};
c ← sX[7];
```

- **sl1** sX
  - shift a register left 1 bit and shift 1 into the LSB
  - pseudo operation:

```
sX ← {sX[6:0], 1};
c ← sX[7];
```

- **slx** sX
  - shift a register left 1 bit and shift sX[0] into the LSB
  - pseudo operation:

```
sX ← {sX[6:0], sX[0]};
c ← sX[7];
```

- **sla** sX
  - shift a register left 1 bit and shift c into the LSB
  - pseudo operation:

```
sX ← {sX[6:0], c};
c ← sX[7];
```

- **rl** sX
  - rotate a register left 1 bit
  - pseudo operation:

```
sX ← {sX[6:0], sX[7]};
c ← sX[7];
```

- **rr** sX
  - rotate a register right 1 bit
  - pseudo operation:

```
sX ← {sX[0], sX[7:1]};
c ← sX[0];
```

Data Movement Instructions:

- *Between registers*: the **load** instruction
- *Between a register and data RAM*: the **fetch** and **store** instructions
- *Between a register and an I/O port*: the **input** and **output** instructions

- **load** sX, sY
  - move data between two registers
  - pseudo operation:

    ```
    sX ← sY;
    ```

- **load** sX, KK
  - move a constant to a register
  - pseudo operation:

    ```
    sX ← KK;
    ```

- **fetch** sX, (sY) (**fetch** sX, sY)
  - move data from the data RAM to a register
  - pseudo operation:

    ```
    sX ← RAM[(sY)];
    ```

- **fetch** sX, SS
  - move data from the data RAM to a register
  - pseudo operation:

    ```
    sX ← RAM[SS];
    ```

- **store** sX, (sY) (**store** sX, sY)
  - move data from a register to the data RAM
  - pseudo operation:

    ```
    RAM[(sY)] ← sX;
    ```

- **store** sX, SS
  - move data from a register to the data RAM
  - pseudo operation:

    ```
    RAM[SS] ← sX;
    ```

- **input** sX, (sY) (**in** sX, sY)
  - move data from the input port to a register
  - pseudo operation:

```
port_id ← sY;
sX ← in_port;
```

- **input** sX, KK (**in** sX, KK)
  - move data from the input port to a register
  - pseudo operation:

```
port_id ← KK;
sX ← in_port;
```

- **output** sX, (sY) (**out** sX, sY)
  - move data from a register to the output port
  - pseudo operation:

```
port_id ← sY;
out_port ← sX;
```

- **output** sX, KK (**out** sX, KK)
  - move data from a register to the output port
  - pseudo operation:

```
port_id ← KK;
out_port ← sX;
```

PORT_ID is an 8-bit address for the MUX. It selects which port is shorted to IN_PORT[7:0]. The following figure shows the coceptual diagram for the IN_PORT and the PORT_ID port.

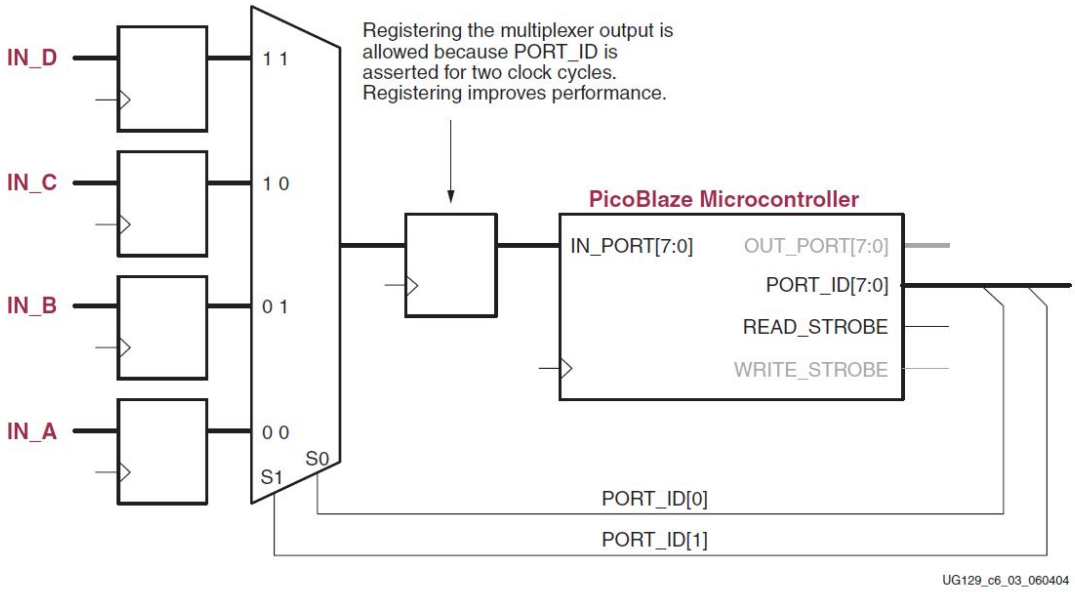


Figure 6-3: **Multiplex Multiple Input Sources to Form a Single IN_PORT Port**

The output diagram is as follows:

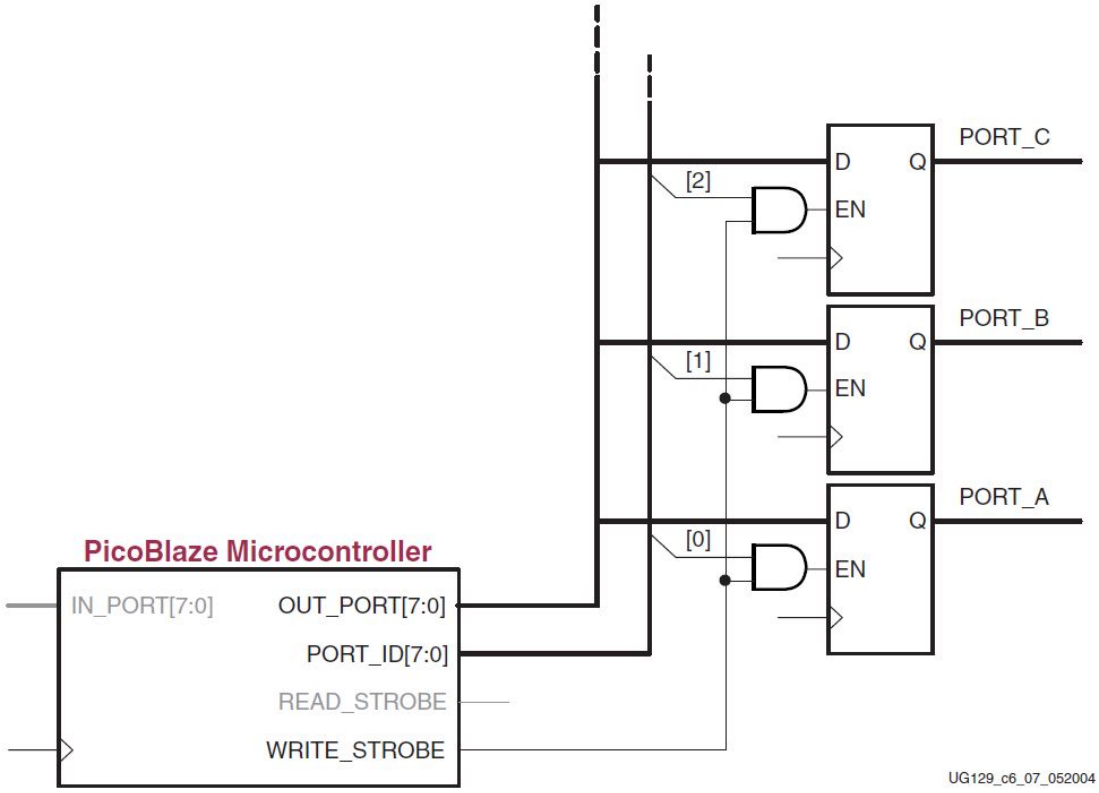


Figure 6-7: **Simple Address Decoding for Designs with Few Output Destinations**

Program Flow Control Instructions:

In PicoBlaze, the program counter indicates where to fetch the instruction. By default, the execution proceeds to the next address in the instruction memory and the program counter is incremented implicitly (i.e., pc t pc + 1).

The **jump, call, and return** instructions can explicitly load a value to the program counter and modify the program flow. These instructions can be executed unconditionally or conditionally based on the values of the carry and zero flags. A jump instruction loads a new value to the program counter if the corresponding condition is met. The program execution changes the regular flow and branches to the new address. The program flow continues normally after this point. The mnemonics, brief descriptions, and pseudo operations of these instructions are shown below. Recall that AAA is for the 10-bit instruction memory address and pc is for the program counter.

- **jump** AAA
  - unconditionally jump
  - pseudo operation:

```
pc ← AAA;
```

- **jump c**, AAA
  - jump if the carry flag is set
  - pseudo operation:

```
if c==1 then pc ← AAA else pc ← pc + 1;
```

- **jump nc**, AAA
  - jump if the carry flag is not set
  - pseudo operation:

```
if c==0 then pc ← AAA else pc ← pc + 1;
```

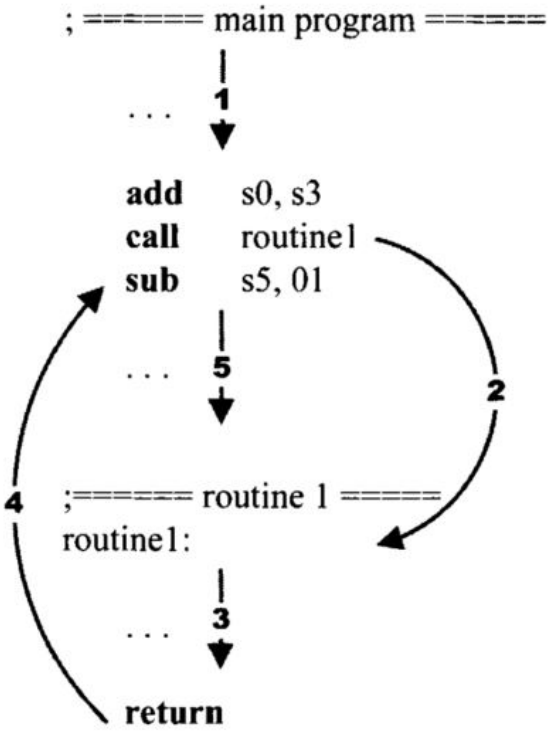
- **jump z**, AAA
  - jump if the zero flag is set
  - pseudo operation:

```
if z==1 then pc ← AAA else pc ← pc + 1;
```

- **jump nz**, AAA
  - jump if the zero flag is not set
  - pseudo operation:

```
if z==0 then pc ← AAA else pc ← pc + 1;
```

The **call and return** instructions are used to implement a software function. When a function is called, the processor suspends the current execution and branches to the corresponding routine. When the routine computation is completed, the processor returns to the suspended point and continues the execution. Like a jump instruction, a call instruction loads a new value to the program counter if the corresponding condition is met. In addition, it also saves the current value of the program counter in a special buffer, known as the stack. The new address represents the starting point of a routine. The routine should include a return instruction in the end.



PicoBlaze allows nested function calls, which means that a function can be called within another function. To support this feature, a stack, which is a last-in-first-out buffer, is used to store the program counter's values. In this buffer, the address of the newest call is pushed to the top of the stack (i.e., the "last-in"). Assume that this routine does not contain other function call inside. It will be completed first and the saved returned address is on the top of the stack. It should be popped from the stack (i.e., "first-out") to resume the previous execution. PicoBlaze provides a 3 1 -word stack for the nested call and return operations. The mnemonics, brief descriptions, and pseudo operations of the call and return in- structions are shown below. Recall that t o s is for the top-of-stack pointer. The STACK [ ] notation represents the content of the stack.

- **call** AAA
  - unconditional call subroutine
  - pseudo operation:

```
tos ← tos + 1;
STACK[tos] ← pc;
pc ← AAA;
```

- **call c**, AAA
  - call subroutine if the carry flag is set
  - pseudo operation:

```
if c==1 then
   tos ← tos + 1;
   STACK[tos] ← pc;
   pc ← AAA;
else
```

```
pc ← pc + 1;
```

- **call nc**, AAA
  - call subroutine if the carry flag is not set
  - pseudo operation:

```
if c==0 then
   tos ← tos + 1;
   STACK[tos] ← pc;
   pc ← AAA;
else
   pc ← pc + 1;
```

- **call z**, AAA
  - call subroutine if the zero flag is set
  - pseudo operation:

```
if z==1 then
   tos ← tos + 1;
   STACK[tos] ← pc;
   pc ← AAA;
else
   pc ← pc + 1;
```

- **call nz**, AAA
  - call subroutine if the zero flag is not set
  - pseudo operation:

```
if z==0 then
   tos ← tos + 1;
   STACK[tos] ← pc;
   pc ← AAA;
else
   pc ← pc + 1;
```

- **return (ret)**
  - unconditional return
  - pseudo operation:

```
pc ← STACK[tos] + 1;
tos ← tos - 1;
```

- **return c (ret c)**
  - return if the carry flag is set
  - pseudo operation:

```
if c==1 then
   pc ← STACK[tos] + 1;
   tos ← tos - 1;
else
   pc ← pc + 1;
```

- **return nc (ret nc)**
  - return if the carry flag is not set
  - pseudo operation:

```
if c==0 then
   pc ← STACK[tos] + 1;
   tos ← tos - 1;
```

```
else
   pc ← pc + 1;
```

- **return z (ret z)**
  - return if the zero flag is set
  - pseudo operation:

```
if z==1 then
   pc ← STACK[tos] + 1;
   tos ← tos - 1;
else
   pc ← pc + 1;
```

- **return nz (ret nz)**
  - return if the zero flag is not set
  - pseudo operation:

```
if z==0 then
   pc ← STACK[tos] + 1;
   tos ← tos - 1;
else
   pc ← pc + 1;
```

Interrupt Related Instructions:

Interrupt is another mechanism to alter program execution and its detail is discussed in Chapter 18. Unlike the jump and call instructions, it is initiated from an external request. When the interrupt flag is enabled and the interrupt request is asserted, PicoBlaze completes execution ofthe current instruction, saves the address ofthe next instruction in the call/return stack, preserves the cany and zero flags, disables the interrupt flag, and loads the program counter with 3FF, which is the starting address of the interrupt service routine. PicoBlaze has two return-from-interrupt instructions, which resume operation from the interrupted location. It also has two instructions that enable and disable the interrupt request by setting or clearing the interrupt flag, i. The mnemonics, brief descriptions, and pseudo operations of these instructions are:

- **returni disable (reti disable)**
  - return from interrupt service routine and keep the interrupt flag disabled
  - pseudo operation:

```
pc ← STACK[tos];
tos ← tos - 1;
i ← 0;
c ← preserved c;
z ← preserved z;
```

- **returni enable (reti enable)**
  - return from interrupt service routine and keep the interrupt flag enabled
  - pseudo operation:

```
pc ← STACK[tos];
tos ← tos - 1;
i ← 1;
c ← preserved c;
z ← preserved z;
```

- **enable interrupt (eint)**
  - enable interrupt request
  - pseudo operation:

```
i ← 1;
```

- **disable interrupt (dint)**
  - disable interrupt request
  - pseudo operation:

```
i ← 0;
```

Assembler Directives:

An assembler directive looks like an instruction in an assembly program. However, it is not part of the microcontroller's instruction set but is used to help program development. As its name suggests, a directive "directs" the assembler to perform a specific task, such as defining a constant or reserving data space.

15.6.1 The KCPSM3 directives

The mnemonics, descriptions, and examples of key directives used in the KCPSM3 assembler are:

- **address**
  - The directive specifies the subsequent code to be put to a specific address in the instruction ROM.
  - Example:

```
address  3FF
```

- **namereg**
  - The directive gives a symbolic name for a register. It makes code more descriptive.
  - Example:

```
namereg  s5, index
```

- **constant**
  - The directive gives a symbolic name for a constant. It makes code more descriptive.
  - Example:

```
constant  max, F0
```

2. PicoBlaze Assembly Code Development

The following code segment shows how to set, clear, and toggle the second LSB of the SO register:

```
constant SET_MASK, 02    ;mask=0000_0010
constant CLR_MASK, FD    ;mask=1111_1101
constant TOG_MASK, 02    ;mask=0000_0010

or  s0, SET_MASK     ;set 2nd LSB to 1
and s0, CLR_MASK     ;clear 2nd LSB to 0
xor s0, TOG_MASK     ;toggle 2nd LSB
```

The toggle operation is based on the observation that for any Boolean variable x, $x \oplus 0 = x$ and $x \oplus 1 = x'$. The same principle can be applied to multiple bits. For example, we can clear the upper nibble (i.e., four MSBs) by using

```
and s0, 0F          ;mask=0000_1111
```

We can also apply the concept of the and mask to the **test** instruction to check a single bit. For example, the following code segment tests the MSB of the s0 register and branches to a proper routine accordingly:

```
    test s0, 80          ;mask=1000_0000
    jump nz, msb_set     ;MSB is 1, branch to msb_set
    ;code for MSB not set
    jump  done
msb_set:
    ;code for MSB set
    ...
done:
    ...
```

A single bit can be extracted by applying the previous code. For example, the following code segment extracts the MSB of the s0 register and stores it in the s1 register:

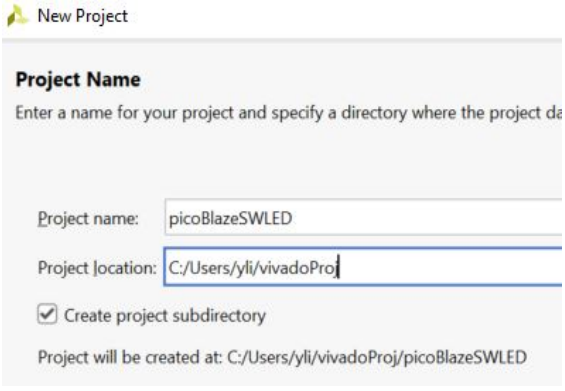
```
    load s1, 00
    test s0, 80       ;mask=1000_0000, extract MSB
    jump z, done      ;yes, MSB is 0
    load s1, 01       ;no, load 1 to s1


done:
    ...
```

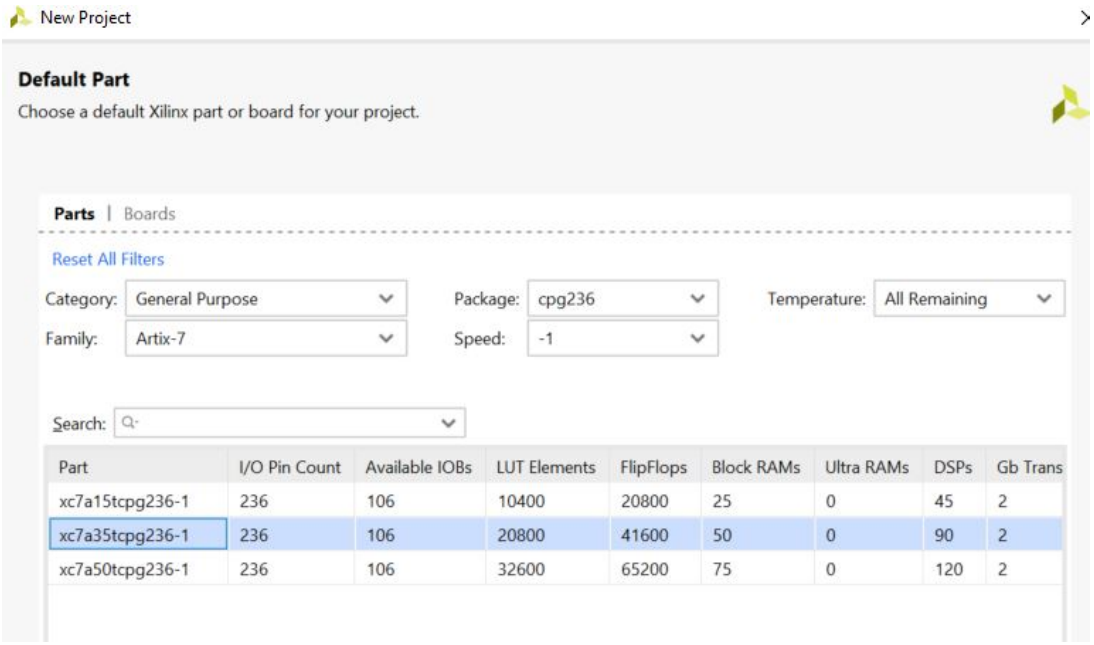
3. The workflow of SoC design using KCPSM6 and Basys 3

Download the KCPSM6_Release9_30Sept14.zip file here:
https://www.xilinx.com/products/intellectual-property/picoblaze.html#design

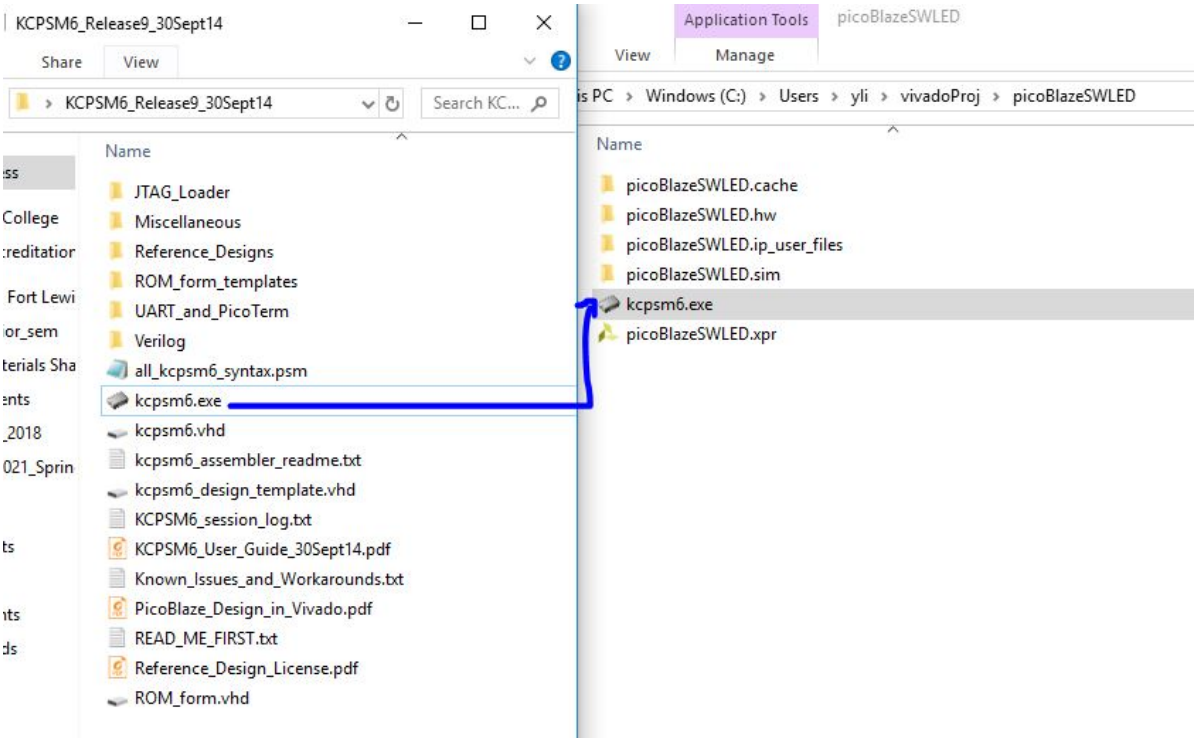
KCPSM - constant (K) coded programmable state machine. The KCPSM6 module is the PicoBlaze processor.

Create a new project in Vivado.

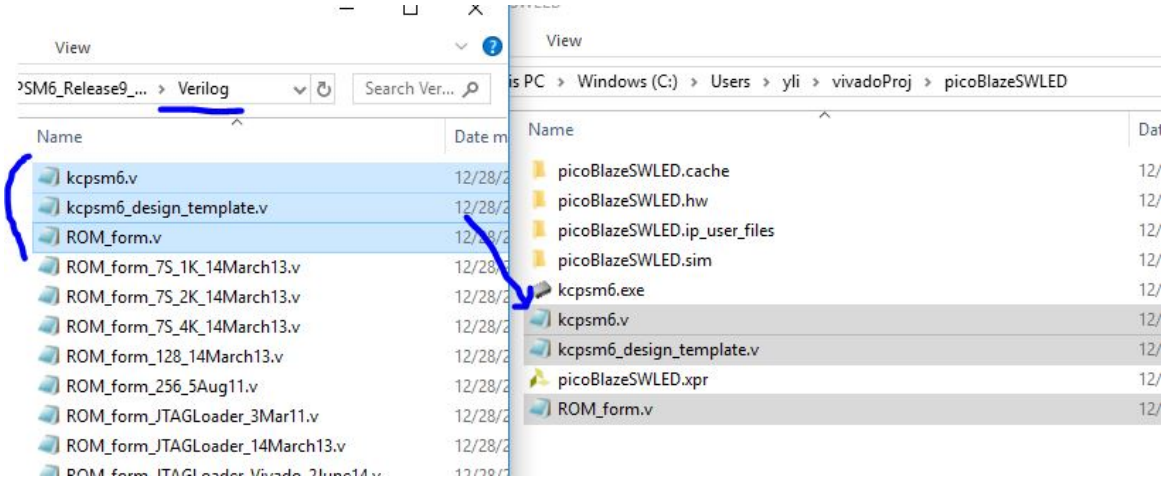




Copy the assembler kcpsm6.exe to the project folder. This exe file is used to convert the assemly conde into a ROM.v code for your project.



In the Verilog subfolder, copy kcpsm6.v, kcpsm6_design_template.v, and ROM_form.v to the project folder. kcpsm6.v is the HDL of the PicoBlaze core; kcpsm6_design_template.v is not a desiign file, it contains examples and code blocks for your design; ROM_form.v determines the compiled ROM file to be a .v file but not a .vhd file. If you program using VHDL, you need the ROM_form.vhd to be in this folder.



Create a main.v file as the top level of the design, copy/paste the following code blocks from the kcpsm6_design_template.v file to your main.v file.

```
wire    [11:0]  address;
wire    [17:0]  instruction;
wire                    bram_enable;
wire    [7:0]           port_id;
wire    [7:0]           out_port;
reg     [7:0]           in_port;
wire                    write_strobe;
wire                    k_write_strobe;
wire                    read_strobe;
reg                     interrupt;          //See note above
wire                    interrupt_ack;
wire                    kcpsm6_sleep;       //See note above
reg                     kcpsm6_reset;       //See note above

//
// Some additional signals are required if your system also needs to reset KCPSM6.
//

wire                    cpu_reset;
wire                    rdl;

//
// When interrupt is to be used then the recommended circuit included below requires
// the following signal to represent the request made from your system.
//

wire                    int_request;
```

```
kcpsm6 #(
      .interrupt_vector       (12'h3FF),
      .scratch_pad_memory_size(64),
      .hwbuild                (8'h00))
processor (
      .address                (address),
      .instruction    (instruction),
      .bram_enable    (bram_enable),
      .port_id                (port_id),
      .write_strobe   (write_strobe),
      .k_write_strobe         (k_write_strobe),
      .out_port               (out_port),
      .read_strobe    (read_strobe),
      .in_port                (in_port),
      .interrupt              (interrupt),
      .interrupt_ack  (interrupt_ack),
      .reset          (kcpsm6_reset),
      .sleep          (kcpsm6_sleep),
      .clk                    (clk));
```

SW
1'b0
rdl
1'b0

use your Rom file's name

```
<your_program> #(
      .C_FAMILY                 ("V6"),     //Family 'S6' or 'V6', 7S for the 7-series
      .C_RAM_SIZE_KWORDS      (2),    //Program size '1', '2' or '4'
      .C_JTAG_LOADER_ENABLE   (1))    //Include JTAG Loader when set to '1'
  program_rom (                               //Name to match your PSM file
      .rdl                    (kcpsm6_reset),
      .enable                 (bram_enable),
      .address                (address),
      .instruction    (instruction),
      .clk                    (clk));
```

rdl

For the behavioral part, you can use the following code:

```
always @(posedge clk)
       if(write_strobe ==1'b1)
                     led_reg <= out_port;
   assign led = led_reg;
```

Now, the only missing piece of the main.v file is the port declaration, please complete it on your own.

The purpose of this project is to use switches [7:0] to turn on/off the corresponding leds. The following program was named as prog.psm. You can use gvim to edit it.
The following script receives sw inputs from in_port number 0 and send it directly to the output 'out_port' number 0 as well.

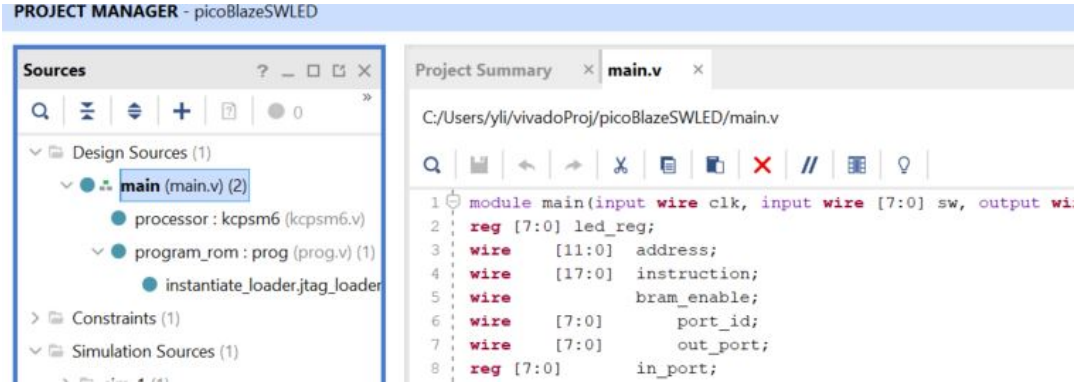
```
3 LOOP:
4         input s0,00
5         output s0,00
6         jump LOOP
7
```

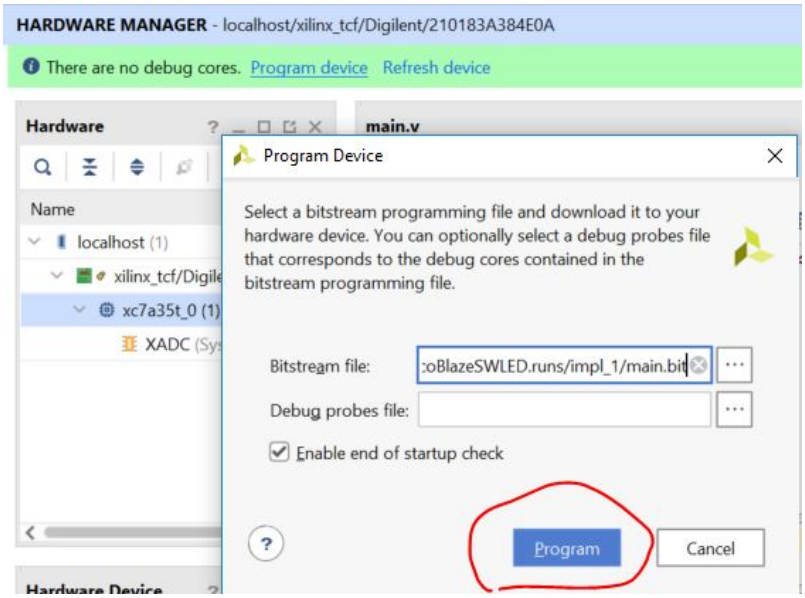
Save this file in the project folder as "prog.psm' (important). In command line, enter the folder using 'cd <PATH> to the folder', then use 'kcpsm6.exe prog.psm'.

```
C:\Users\yli\vivadoProj\vahidExample>cd C:\Users\yli\vivadoProj\picoBlazeSWLED

C:\Users\yli\vivadoProj\picoBlazeSWLED>kcpsm6.exe prog.psm
```

It generates the ROM.v file for the project.

Add these design sources to the project and run synthesis, implementation, and generate bitstream respectively.





Download the bitstream file to your Basys 3 board, you should be able to see the switches control the LEDs.

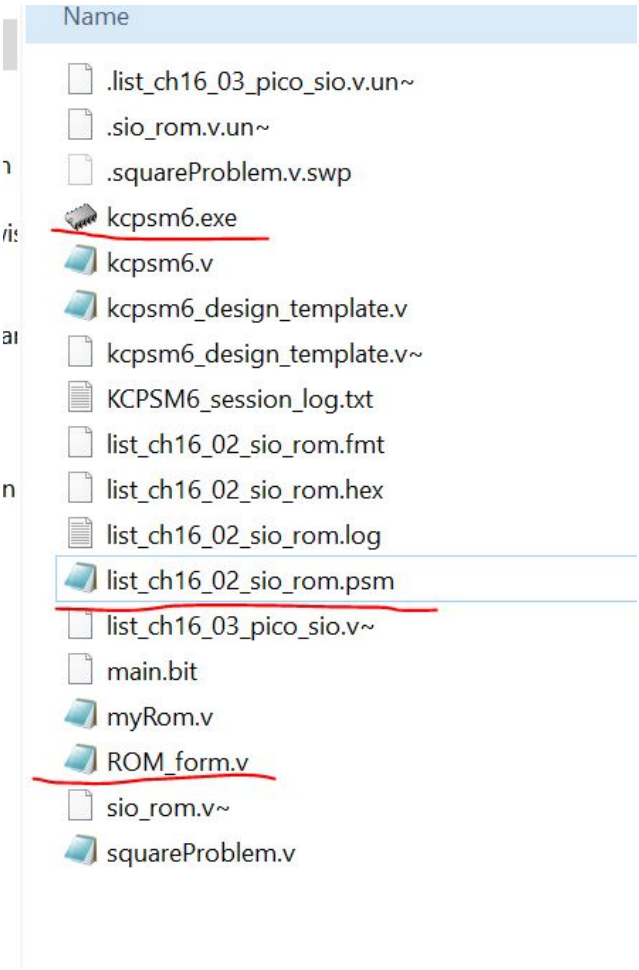
Yiyan Li

Watch on

4. The Square Problem

There is an interesting problem in Chapter 16 of Pong Chu's textbook - The square problem. Please read it through. (PDF)

To implement it on your Basys 3 board using the KCPSM6 compiler you need to do the following:
1. The assembly code is compatible with both KCPSM3 and KCPSM6 so directly put it in the same directory with kcpsm6.exe and ROM_form.v: (the three files)



Then compile it in your command window

```
kcpsm6.exe list_ch16_02_sio_rom.psm
```

The top module in Pong Chu's book doesn't work for KCPSM6. There are many issues with it, for example, the instruction address should be 12 bits but not 10 bits. It should be 'wire [11:0] instruction'. There are many other issues so I recommend you use the kcpsm6 design template for the port declaration. Definitely use kcpsm6.v as the CPU core.

Listing 16.3 PicoBlaze with a simple I/O configuration

```
module pico_sio
   (
    input wire clk, reset,
    input wire [7:0] sw,
    output wire [7:0] led
   );

   // signal declaration
   // KCPSM3/ROM signals
   wire [9:0] address;
   wire [17:0] instruction;
   wire [7:0] port_id, in_port, out_port;
   wire  write_strobe;
   // register signals
   reg [7:0] led_reg;
```



```
   //body
   // ============================================================
   //  KCPSM and ROM instantiation
   // ============================================================
   kcpsm3 proc_unit
      (.clk(clk), .reset(reset), .address(address),
       .instruction(instruction), .port_id(),
       .write_strobe(write_strobe), .out_port(out_port),
       .read_strobe(), .in_port(in_port), .interrupt(1'b0),
       .interrupt_ack());
   sio_rom rom_unit
      (.clk(clk), .address(address),
       .instruction(instruction));
   // ============================================================
   //  output interface
   // ============================================================
   always @(posedge clk)
      if (write_strobe)
         led_reg <= out_port;
   assign led = led_reg;
   // ============================================================
   //  input interface
   // ============================================================
   assign in_port = sw;

endmodule
```

In the end of the top module, you can use the following code:

```
    always @(posedge clk)
       if (write_strobe)
          led_reg <= out_port;
    assign led = led_reg;
    assign in_port = sw;
endmodule
```

It uses the 'assign' funcion so on the top of the module when you declare in_port, it should be 'wire' but not 'reg'.

Here is the demonstration of the implementation:

Yiyan Li

Watch on

5. Assembly example: adding a 0x13 to the LEDs

The following program load 0x13 to the input port and send 0x13 to the LEDs.

```
1 ; show a 0x13 on the LED
2         INPUT s0,00
3         load s1,s0
4         add s1,13
5         OUTPUT s1,00
6
```

Here is the result:

Yiyan Li

Watch on

6. Assembly example: use the MASK

When you rerun the assembly file, please do the following to avoid potential glitches: (this applies to all the following tasks)
1. delete the prog.v file (the instruction memory file) before you recompile the new assembly code.
2. reload the prog.v file in your Vivado (remove the current one, and add the new one from add design sources).

```
 8 ; the last two bits of the LED will be always 11
 9         load s1,00 ; clear s1
10         constant lowerMASK, 03; 0000 0011
11         input s0,00
12         load s1, s0
13         or s1, lowerMASK
14         output s1,00
15
```

The result:

Yiyan Li

Watch on

7. Assembly example: shift the sw input to the right by one bit

```
16 ; shift the input byte to the right by one bit
17         input s0,00
18         load s1,s0
19         sr0 s1
20         output s1,00
21
```

Results:

Yiyan Li

Watch on

8. Assembly example: use jump, return, and load instructions

```
22 ; nz output 6, z output 5
23 FOREVER:
24         input s0,00
25         load s2,05
26         load s3,06
27         load s1,s0
28         test s1,01
29         jump nz, TESTING
30         output s2,00
31         jump FOREVER
32 TESTING:
33         output s3,00
34         return
35
```

It doesn't acturally return to "output s2, 00" after the jump. So the 'jump' instruction will skip the following instructionss. Try to use 'call' in stead of 'jump' for line 29 and see if it changes the result.

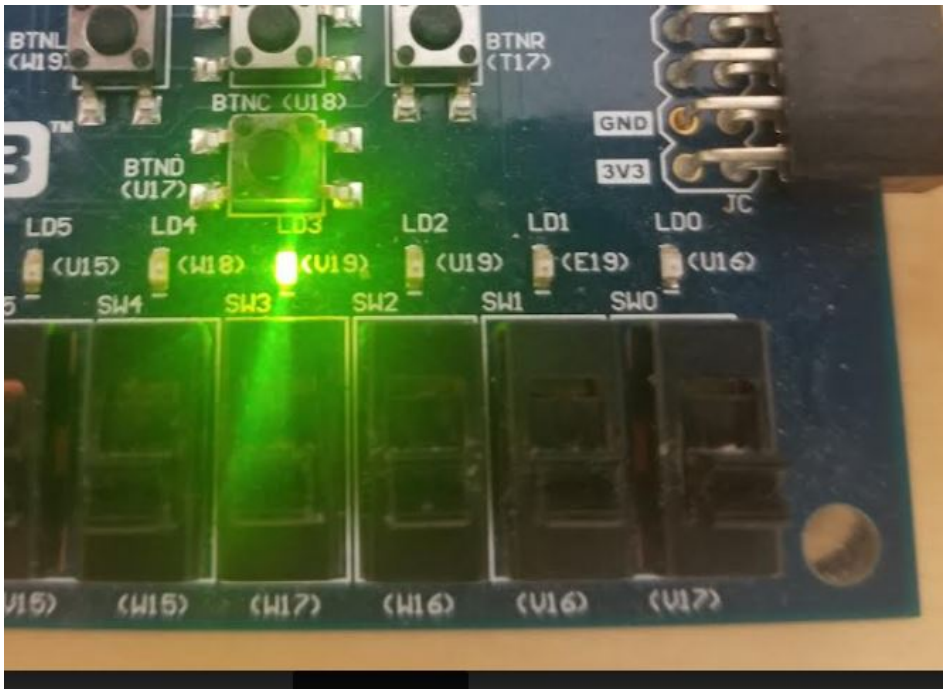
Yiyan Li

Watch on

---

9. Assembly example: practice with multiple subroutines

```
65 ; when the counter reduces by 1, LED incremments by 1
66         load s2,08
67         load s3,00
68 FOREVER:
69         test s2,ff
70         jump nz,NOTZ
71         jump z,ISZ
72 NOTZ:
73         sub s2,01
74         add s3,01
75         jump FOREVER
76 ISZ:
77         output s3,00
78         jump FOREVER
79
```



10. Assembly exapmle: use the 'compare' instruction instead

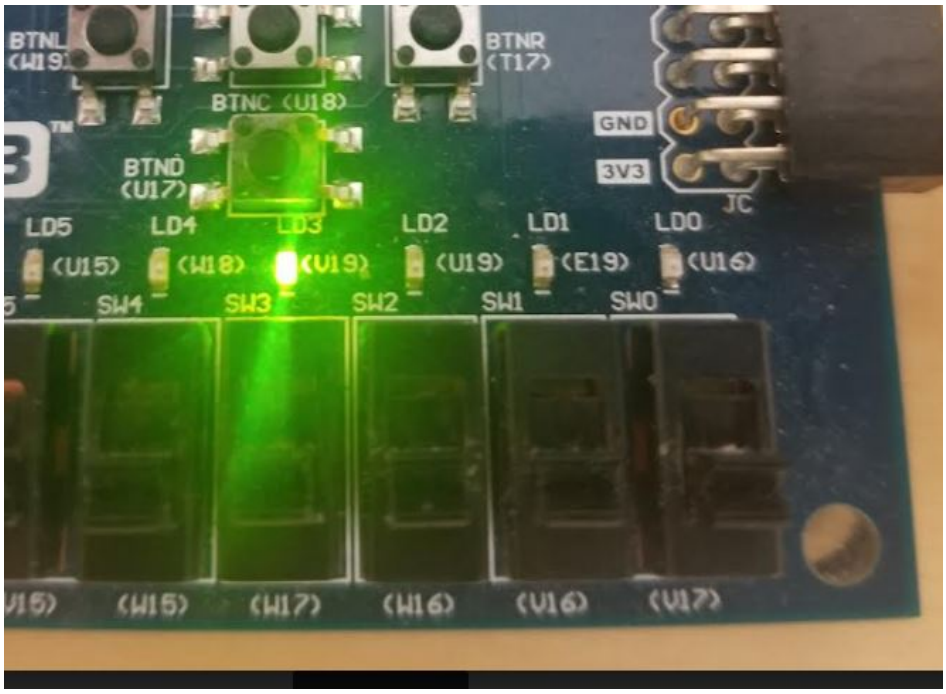
Given the following explanation of the 'compare' instruction, revise the assembly code above to achieve the same result.

- **compare** sX, sY (**comp** sX, sY)
  - compare two registers and set the flags
  - pseudo operation:

```
if sX==sY then z ← 1 else z ← 0;
if sY>sX then c ← 1 else c ← 0;
```

- **compare** sX, KK (**comp** sX, KK)
  - compare a register and a constant and set the flags
  - pseudo operation:

```
if sX==KK then z ← 1 else z ← 0;
if KK>sX then c ← 1 else c ← 0;
```



11. Assembly programming task - design an assembly code to show the number of switches that are ON on the LEDs in the binary form.
The results must be able to show on LEDs without reprogramming the FPGA.

Demo video:

Yiyan Li

Watch on

Tasks:

Repeat the work in Section 3 - 11 in this tutorial. If the Section asks you to complete something, please complete it. Show the code and demo videos for credits.

Section 3 - 10: 10 points each
Section 11: 20 points

References:

Vahid Meghdadi's YouTube channel
ptracton GitHub link
PicoBlaze Release 7, Release 9
Pong Chu code for book examples
Basys 3 and microblaze, and VGA
Pong Chu's book: FPGA Prototyping by Verilog Examples - Xilinx Spartan - 3 Version